



## **Chapter S2B: Object-Oriented Programming**

### **CONTENT**

| <b>No.</b> | <b>Topic</b>                          | <b>Syllabus Guide</b> |
|------------|---------------------------------------|-----------------------|
| <b>1</b>   | <b>Introduction to OOP</b>            | <b>2.5</b>            |
| <b>2</b>   | <b>Classes and Objects</b>            | <b>2.5.1</b>          |
| 2.1        | Class Diagram Format                  | 2.5.5                 |
| 2.2        | Instantiation                         |                       |
| <b>3</b>   | <b>Encapsulation</b>                  | <b>2.5.2</b>          |
| 3.1        | Private Attributes and Public Methods |                       |
| 3.2        | Getters and Setters                   |                       |
| <b>4</b>   | <b>Inheritance</b>                    | <b>2.5.3</b>          |
| 4.1        | The “Is-a” Rule                       |                       |
| 4.2        | Benefits of Inheritance               |                       |
| <b>5</b>   | <b>Polymorphism</b>                   | <b>2.5.4</b>          |
| 5.1        | Method Overriding                     |                       |
| 5.2        | Benefits of Polymorphism              |                       |
| <b>6</b>   | <b>Implementation in Python</b>       |                       |
| 6.1        | Example: Rectangle and Square class   |                       |
| 6.2        | Example: Person and Student class     |                       |

## 1. Introduction to Object-Oriented Programming

Imagine you are building a video game. Every character, every car, every building you see – these are objects that have their own properties and actions. Object-oriented programming (OOP) is a powerful way to organise programs by thinking about them as a world full of interacting objects.

In OOP, the world inside a program is viewed as a collection of interacting objects. An object can represent a real-world entity like a person, animal, place, event, or even something abstract like a bank account or a data structure (e.g. linked list, stack, or queue).

In an object-oriented program, each object is responsible for:

- Managing its own data (attributes)
- Handling its own actions (methods)

Objects communicate with one another by sending messages and receiving responses. All processing in the program is performed through objects.

### Object Attributes and Behaviours

An object has **attributes** that describe itself. A car may have attributes like *make*, *engine size*, and *colour*. A person may have *first name*, *last name*, and *date of birth*.

An object also has a **state**, which describes its current condition. For example, a radio can be on or off, tuned to a particular station, and set to a specific volume. A bank account may have a certain account type and an account balance.

An object has **behaviours** which are actions that an object can perform (methods). For example, a cow can stand, walk, moo, eat and drink. A linked list can add a node, delete a node, display data and perform other actions.

## 2. Classes and Objects

If objects are the characters and props inside a video game, then classes are the blueprints used to build them. A class defines the **attributes** (data) and **behaviours** (methods) that its objects will have. You can think of a class like a cookie cutter – it shapes cookies (objects) but the cookies themselves are individual and unique.

An **attribute** is a piece of data stored in an object. It is also known as property or data member of the class. For example, a **Car** class might have attributes like *make*, *engine size* and *colour*. A **Person** class might have *first name*, *last name* and *date of birth*.

A **method** is an action that an object can perform. For example, a **Bank Account** class might have methods like **deposit()** and **withdraw()**.

## Example: AccountHolder Class

Imagine a bank needs to manage its customers. It might create a class called **AccountHolder** with:

- **Attributes:**
  - HolderName: String
  - Address: String
  - Profession: String
  - Birthday: Date
- **Methods:**
  - ChangeResidence (newAddress: String)
  - ChangeProfession (newProfession: String)

### 2.1 Class Diagram Format

We often draw classes using a simple diagram with three sections:

| Class Name                                                 |
|------------------------------------------------------------|
| <b>Attributes</b><br>- <attribute 1>: data type<br>- ..... |
| <b>Methods</b><br>+ <method 1><br>+ .....                  |

Example:

| AccountHolder                                                                         |
|---------------------------------------------------------------------------------------|
| - HolderName: String<br>- Address: String<br>- Profession: String<br>- Birthday: Date |
| + ChangeResidence (newAddress: String)<br>+ ChangeProfession (newProfession: String)  |

Write Class Name in singular form.

Symbols used:

- **+** (Public) – can be accessed from anywhere.
- **-** (Private) – can only be accessed inside the class.
- **#** (Protected) – can be accessed by the class and its child classes.

For this A-level syllabus, we mainly focus on **public methods** and **private attributes**.

### Pseudocode Example:

```
CLASS AccountHolder

    PRIVATE HolderName: STRING
    PRIVATE Address: STRING
    PRIVATE Profession: STRING
    PRIVATE Birthday: STRING

    PUBLIC PROCEDURE new(name,address,profession,birthday: STRING)
        (instructions to set initial values)
    ENDPROCEDURE

    PUBLIC PROCEDURE ChangeResidence (newAddress: STRING)
        (instructions)
    ENDPROCEDURE

    PUBLIC PROCEDURE ChangeProfession (newProfession: STRING)
        (instructions)
    ENDPROCEDURE

ENDCLASS
```

This shows that **instance variables (or attributes)** are usually private, and **methods** are public to protect the data inside the class.

## 2.2 Instantiation (Create an Object)

Once we have a class, we can use it to create **objects**.

Creating an object is called **instantiation**. When we instantiate an object:

- The object is given its own copy of the attributes and methods defined by the class.
- The attributes can hold different values for different objects.

### Constructors

A **constructor** is a special method that runs when an object is created. Its job is to **set up** the new object, often by assigning initial values to its attributes.

In pseudocode, a constructor often has the name **new**.

Example:

```
PUBLIC PROCEDURE new(name,address,profession,birthday: STRING)
    (instructions to set initial values)
ENDPROCEDURE
```

Some languages also have **destructors** to clean up an object when it is no longer needed. (Python handles memory automatically, so we usually don't need destructors.)

### Example: Creating an AccountHolder Object

Suppose we have already defined the **AccountHolder** class.

Now we want to create an actual customer called **acct1**.

We would instantiate it like this:

```
acct1= new AccountHolder("Ronald Lee","21 Kallang","Teacher","12/12/1988")
```

- **acct1** is called a reference type variable – it holds a reference (or pointer) to the object stored in memory.
- The class name **AccountHolder** is used like a function to create the object.

Each object created from a class is called an **instance**.

### Instances

All objects from the same class share the same structure (same attributes and methods).

Each object has its own unique data (attribute values).

For example, each object stores its **own** name, address, profession and birthday:

| Object | Class         | HolderName | Address     | Profession | Birthday   |
|--------|---------------|------------|-------------|------------|------------|
| acct1  | AccountHolder | Ronald Lee | 21 Kallang  | Teacher    | 12/12/1988 |
| acct2  | AccountHolder | Jane Tan   | 45 Lavender | Engineer   | 07/07/2000 |

## 3. Encapsulation

Imagine you want to buy a drink from the vending machine. You can press buttons to get drinks, but you cannot see or touch all the wires, motors, and systems inside. This is the idea behind **encapsulation**.

In programming, encapsulation means **grouping** an object's data (attributes) and actions (methods) **together** and **hiding** the internal details from the outside world.

Encapsulation protects an object's data by controlling how it's accessed and changed.

- **Controlled Access:** Data is only accessible through public methods (also called getters and setters). This helps keep the data accurate and consistent.
- **Accident Prevention:** Since other parts of the program can't directly touch the data, it's less likely that bugs or errors will occur from accidental changes.
- **Easier Teamwork:** Makes it easier for different programmers to work on separate parts of a program without interfering with each other. Since each object hides its internal details, others only need to know how to use it, not how it works inside.

This leads to **implementation independence** – the idea that you can change how an object works internally (such as using a different algorithm or data structure) without affecting the rest of the program, as long as the interface (method) stays the same.

This technique of hiding the inner workings of an object and showing only what's necessary is called **information hiding**.

### 3.1 Private Attributes and Public Methods

Usually:

- **Attributes** (instance variables) are marked as **private**.
- **Methods** are **public** so that other classes can interact with the object safely.

Example: In a **Book** object, you might call a method like:

```
myBook1.displayDetails()
```

You don't need to know how **displayDetails()** works internally – just that it will show you the book's information.

### Example: AccountHolder Class

| Scope   | Attributes / Methods                                                                                                                                                    |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Private | HolderName, Address, Profession, Birthday                                                                                                                               |
| Public  | SetResidence(address: String), GetResidence(): String<br>SetProfession(profession: String), GetProfession(): String<br>SetBirthday(birthday: Date), GetBirthday(): Date |

**Attributes are private** – they cannot be accessed directly.

**Methods are public** – they allow safe reading (get) or updating (set) of data.

### 3.2 Getters and Setters

We use:

- **Getter methods** (getSomething()) to **read** attribute values.
- **Setter methods** (setSomething()) to **update** attribute values safely.

They help ensure that only valid data is stored. You can add checks inside setter methods.

Example: Check that birthday is realistic using a function called isValidDate()

```
def setBirthday(self, birthday):
    if isValidDate(birthday): # you can add validation
        self.__birthday = birthday
```

Encapsulation protects the object from bad data. It allows the object's internal code to change without affecting other parts of the program. It keeps the code modular, reusable and easier to debug. Think of it like putting a protective shell around your data!

## 4. Inheritance

Have you ever noticed how children inherit features from their parents?

In programming, **inheritance** works the same way – a **child class** inherits properties and behaviours from a **parent class**.

A **child class** (also called a **subclass/derived class**) can reuse and extend the attributes and methods of a **parent class** (or **superclass/base class**). This allows you to create new classes quickly without rewriting existing code.

## 4.1 The “Is-a” Rule

Before using inheritance, ask: “**Is a** child class truly a **kind of** parent class?”

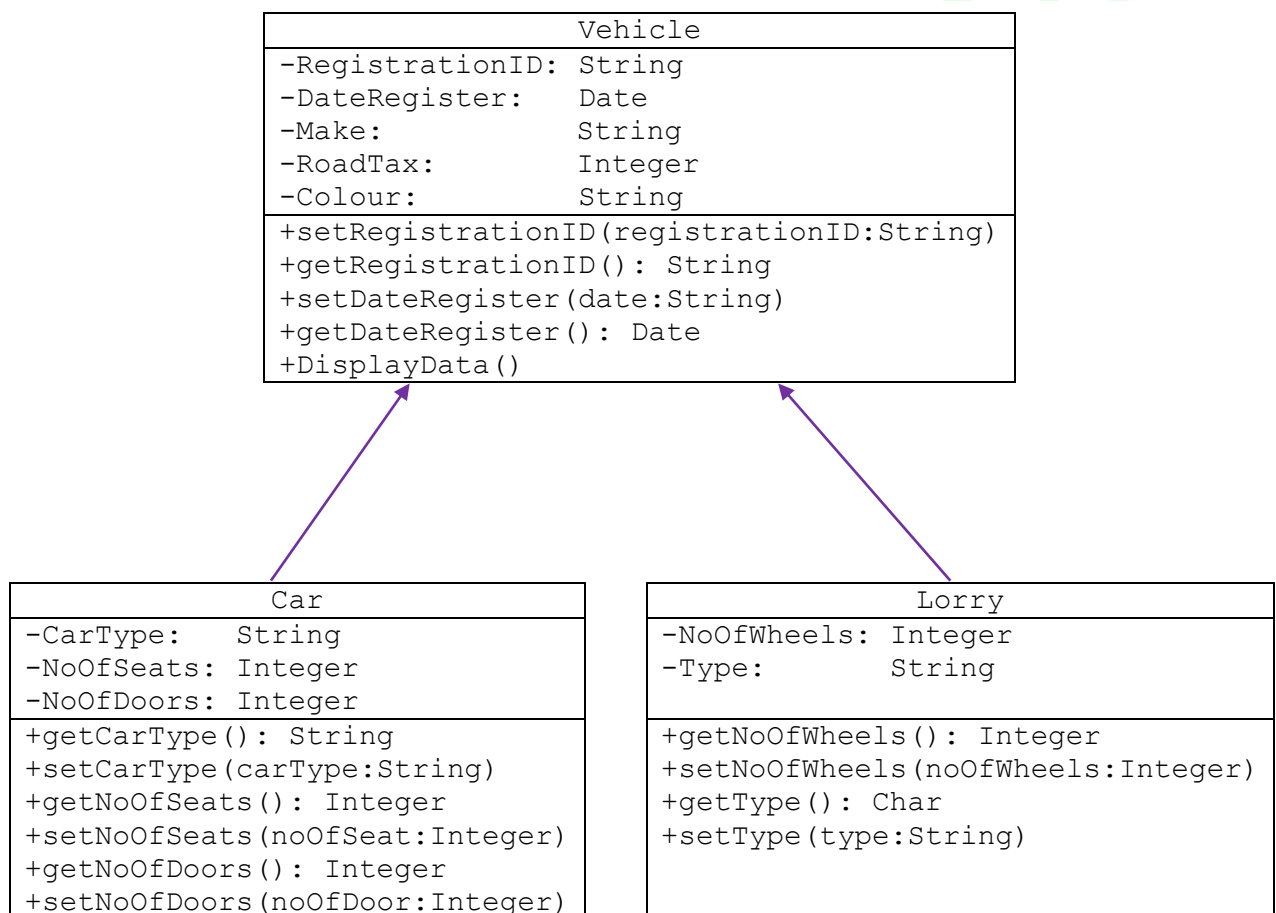
Examples:

- A **Car** is a Vehicle
- A **Lorry** is a Vehicle
- But a **Vehicle** is **not necessarily** a Car

If the “**is-a**” relationship makes sense, then inheritance is appropriate.

**Example: Vehicle, Car, Lorry**

This class diagram shows the superclass Vehicle and the subclasses Car and Lorry in an inheritance relationship.



**Note:** The arrow points upward to the parent class.

There are different kinds of vehicles, e.g. cars and lorries. Though these are different vehicles, they have nevertheless many properties and methods in common. For example, each vehicle has a registration ID, date register, make, road tax payable and colour. And each vehicle has methods related to these attributes, such as `getRegistrationID()`, `DisplayData()`, etc. Both Car and Lorry automatically inherit the Vehicle class attributes and methods.

The derived classes may have their own specific attributes and methods.

**Car** class with additional attributes: `CarType`, `NoOfSeats`, `NoOfDoors`, and additional methods such as `getCarType()`.

**Lorry** class with additional attributes: `NoOfWheels`, `Type` (e.g., light, heavy) and additional methods such as `getNoOfWheels()`.

## 4.2 Benefits of Inheritance

- **Code Reuse**  
No need to rewrite methods like `DisplayData()` for each vehicle type. They can be inherited directly from the parent class.
- **Easier Maintenance**  
Fixing a method in the parent class automatically applies the fix to all child classes, reducing errors and saving time.
- **Logical Structure**  
Models real-world relationships between objects clearly, such as a car or lorry being special types of vehicles.
- **Code Generalisation**  
Allows a derived class to inherit general attributes and methods from a base class, while adding its own specialised features. This reduces code duplication, makes programs easier to maintain and expand, and lays the groundwork for another powerful concept – **polymorphism**, where child classes can override methods to behave differently while still being used in the same general way as their base class.

## 5. Polymorphism

Imagine you press the “play” button – sometimes it plays a song, sometimes it plays a video, depending on what app you’re using. That’s **polymorphism** in action!

In programming, **polymorphism** means “**many forms**.”

It allows different objects to respond differently to the **same action**.

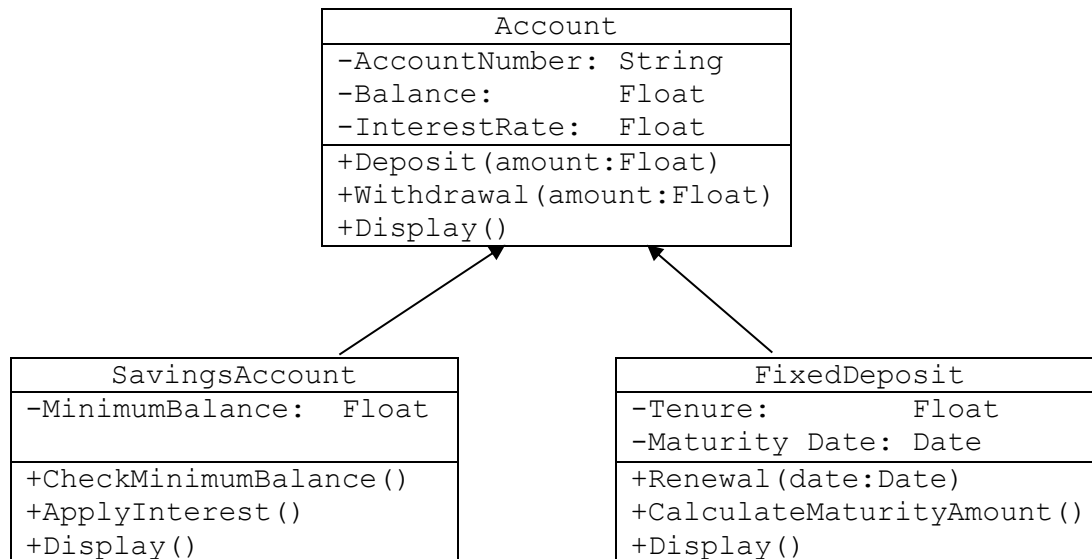
Polymorphism refers to a programming language’s ability to **process objects differently depending on their class**.

Even when different classes share the same method name, the behaviour can change based on the object’s type. This makes programs flexible, easier to maintain and more powerful.

Polymorphism builds on **code generalisation** by allowing subclasses to override the same method in their own unique way, while still using the same method name.

### Example: Account, Savings Account, Fixed Deposit

Suppose we have a base class called `Account` with general attributes like `AccountNumber`, `Balance` and `InterestRate`. Two special kinds of accounts inherit from it: `SavingsAccount` and `FixedDeposit`. Both have their own specific attributes (e.g. `MinimumBalance` for `SavingsAccount`). Even though both classes inherit the `Display()` method, they can **override** it to **show different information**.



### 5.1 Method Overriding

Method overriding happens when a subclass **redefines** a method it inherited from its parent class.

- The method name stays the same.
- The input parameters stay the same.
- The behaviour is different.

#### Example:

- `Account.Display()` shows account number, balance and interest rate.
- `FixedDeposit.Display()` might show the account number, balance, interest rate, tenure and maturity date.

Polymorphism ensures that each class can have its own version of `Display()` without needing to modify existing code. The correct `Display()` method is automatically called based on the object type (`Account`, `SavingsAccount` or `FixedDeposit`)

## 5.2 Benefits of Polymorphism

- **Simplified code**

There is no need to manually check an object's type before calling a method. For example, you can loop through a list of accounts and call `Display()` on each one – the correct version (`SavingsAccount`'s or `FixedDeposit`'s) will automatically run. This reduces the need for complex if-else statements.

- **Greater flexibility**

Different objects can be treated similarly while allowing them to behave differently. For example, when calling `Display()`, a `SavingsAccount` might show minimum balance information, while a `FixedDeposit` might show maturity details.

- **Easier to extend programs**

New object types (such as `CurrentAccount`) can be added without changing existing code. As long as the new class inherits from the same base class and implements the necessary methods, it can fit naturally into the system. This makes the program easier to expand and maintain over time.

Polymorphism allows programs to adapt their behaviour dynamically based on the actual object type at runtime, making the system more flexible and robust.

## 6. Implementation in Python

Python allows programmers to define their own types **using classes**, making programs easier to understand and maintain. Programmers define a **class**, and then create **objects** that are **instances** of the class.

### 6.1 Example: Rectangle and Square Class

The following program shows a definition of a class Rectangle.

```
class Rectangle(object):  
    '''Represent a rectangular section of an image.'''  
  
    def __init__(self, x0, y0, x1, y1):  
        '''Create a rectangle with non-zero area.  
        (x0,y0) is the lower left corner,  
        (x1,y1) the upper right corner.'''  
        self.__x0 = x0  
        self.__x1 = x1  
        self.__y0 = y0  
        self.__y1 = y1  
  
    def area(self):  
        '''Return the area of the rectangle.'''  
        return (self.__x1-self.__x0)*(self.__y1-self.__y0)  
  
    def contains(self, x, y):  
        '''Return True if (x,y) point is inside a rectangle,  
        and False otherwise.'''  
        return (self.__x0 <= x <= self.__x1) and \  
            (self.__y0 <= y <= self.__y1)  
  
    def __str__(self):  
        '''string method to show the coordinates of rectangle  
        (x0,y0) and (x1,y1)'''  
        return "(x0,y0)={},{})\n(x1,y1)={},{})".\  
            format(self.__x0,self.__y0,self.__x1,self.__y1)
```

The keyword **class** signals that we're defining a new type. The **(object)** part says that class **Rectangle** is a kind of object. The documentation string (**docstring**) is used to document the code. Here it describes the **Rectangle** object.

A **constructor** is a method that initialises and sets up a new instance of the object when it is created based on the class. This is done by using the **\_\_init\_\_** method:

```
def __init__(self, x0, y0, x1, y1)
```

In this example, **five** parameters are defined. The first parameter **self** is compulsory and refers to the current object. The next four parameters (**x0**, **y0**, **x1**, **y1**) are optional and used to initialise the values of the attributes.

Leading double underscore is used to declare private attribute. The **four attributes** are declared as **private**, hence defined with leading double underscores. **Public** methods (such as **area** and **contains**) have no leading double underscores. The private attributes can only be accessed through public methods defined in the class.

Two public methods (`area()` and `contains()`) are defined. The `area()` method returns the area of the rectangle, while the `contains()` method tests whether a point is inside the rectangle or not. In the `area()` method, there is no parameter defined except for `self`. In the `contains()` method, `x` and `y` (represent a point) are passed into the method.

The special string method `__str__` returns a string that will be displayed with the built-in `print()` function.

To make a new `Rectangle` object and assign it to a variable called `rec1`:

```
>>> rec1 = Rectangle(0,0,10,20)
```

Using the name of the class as if it were a function tells Python to find the class and build one instance of it. Check that this has worked by printing the value of `rec1`:

```
>>> rec1
<__main__.Rectangle object at 0x000002AB850D0908>
```

Check the type of `Rectangle`:

```
>>> type(Rectangle)
<class 'type'>
```

Use the special string method `__str__` that is defined to display the coordinates:

```
>>> print(rec1)
(x0,y0)=(0,0)
(x1,y1)=(10,20)
```

Use the `area` method to return the area of the rectangle:

```
>>> rec1.area()
200
```

Use the `contains` method to test if a point is inside the rectangle:

```
>>> rec1.contains(5,10)
True
>>> rec1.contains(11,10)
False
```

To see the class definition, use the help method:

```
>>> help(Rectangle)
```

A note on using double underscore to define private attributes:

When the four attributes are defined with double underscores, they are hidden from outside the class (hence private). If you try to access the private attribute directly, an error occurs:

```
>>> rec1.__x1
AttributeError: 'Rectangle' object has no attribute '__x1'
```

But this security can be easily hacked:

```
>>> rec1._Rectangle__x1
10
```

So, using double underscore does not really guarantee privacy, but by using it, you have signalled that you intended it to be private so others who see your code should take note.

Let's define a **Square** class that **inherits** the **Rectangle** class:

```
class Square(Rectangle):
    '''Inherits Rectangle class to represent a square'''

    def __init__(self, x, y, length):
        '''Create a square with (x,y) as lower left corner
        and side as length'''
        super().__init__(x, y, x+length, y+length)
        self.__x = x
        self.__y = y
        self.__length = length
```

The keyword **class** is used to define a new class type with name **Square**. The **(Rectangle)** means that **Square** class inherits all the attributes and methods of **Rectangle** class. This means that a **Square** object has the attributes **\_\_x0**, **\_\_x1**, **\_\_y0**, **\_\_y1**, and methods **area** and **contains** (refer to class definition of **Rectangle**).

In the **Square** class, four parameters (**self**, **x**, **y**, **length**) are defined in the **\_\_init\_\_** constructor. **self** represents the instance of the object, **x** is passed into **self.\_\_x**, **y** is passed into **self.\_\_y**, and **length** is passed into **self.\_\_length**.

The **\_\_init\_\_** defined in **Square** class will override the **\_\_init\_\_** from **Rectangle** class. This means that when an instance of **Square** object is created, we only need to specify **three** arguments (**x**, **y**, **length**) instead of **four** (**x0**, **y0**, **x1**, **y1**).

The **super().\_\_init\_\_(x, y, x+length, y+length)** means that the **Square** instance will call the **constructor of its superclass**. By using **super()**, the **Square** object will call the constructor (**\_\_init\_\_**) from the **Rectangle** class. Hence, the parameters in **Square** class will match the order of parameters defined in **Rectangle** class as shown:

Rectangle: `__init__(self, x0, y0, x1, y1)`

Square: `super().__init__(x, y, x+length, y+length)`

Arrows indicate the mapping: **x0** maps to **x**, **y0** maps to **y**, **x1** maps to **x+length**, and **y1** maps to **y+length**.

```
>>> sq1 = Square(0,0,10)
>>> sq1
<__main__.Square object at 0x000002AB8527E908>
>>> print(sq1)
(x0,y0)=(0,0)
(x1,y1)=(10,10)
>>> sq1.area()
100
>>> sq1.contains(5,10)
True
>>> sq1.contains(11,10)
False
```

Square is called with  
3 arguments: 0, 0, 10

## 6.2 Example: Person and Student Class

Write a class called **Person** that has attributes **name**, **age**, **gender**, **contact** and a method to print these attributes. Add methods to class **Person** to store and retrieve favourite food.

```
class Person(object):

    def __init__(self, name, age=None, gender=None, contact=None):
        #constructor, a Person object may be initialised with name,
        #age, gender and contact may be omitted during instantiation
        '''describe a person with name, age, gender, contact'''
        self.__name = name
        self.__age = age
        self.__gender = gender
        self.__contact = contact

    def setName(self, name):          #modifier
        self.__name = name.title()
    def setAge(self, age):            #modifier
        self.__age = age
    def setGender(self, gender):      #modifier
        self.__gender = gender
    def setContact(self, contact):    #modifier
        self.__contact = contact
    def getName(self):                #accessor
        return self.__name
    def getAge(self):                 #accessor
        return self.__age
    def getGender(self):              #accessor
        return self.__gender
    def getContact(self):             #accessor
        return self.__contact

    def Display(self):
        '''display attributes of Person'''
        try:
            print("Name:", self.__name)
            print("Age:", self.__age)
            print("Gender:", self.__gender)
            print("Contact:", self.__contact)
        except:
            print("Incomplete data for", self.__name)

    def setFavFood(self, food = None):
        '''set favourite food, default food is None'''
        if food == None:
            food = input("{} , what's your favourite food?".format(self.__name))
        self.__favFood = food

    def getFavFood(self):
        '''return favourite food'''
        return self.__favFood
```

Write another class called **Student** that inherits **Person** class with additional attribute **school**. Add methods in class **Student** to

- store the student's school and display the school,
- store and retrieve pocket money.

```
class Student(Person):
    '''inherits Person class with additional school attribute'''
    def __init__(self, name):
        #constructor, only name needs to be initialised
        self.__school = "" #initialise school to empty string
        self.setName(name)

    def setSchool(self):
        '''set school of student'''
        school = input("{} , what's your school?".\
                        format(self.getName()))
        self.__school = school

    def getSchool(self):
        '''get school of student'''
        return self.__school

    def Display(self):
        '''display attributes of Student'''
        try:
            super().Display()
            print("School:", self.__school)
        except:
            print("Incomplete data for",self.getName())

    def setPocketMoney(self, money = None):
        '''set pocket money of student'''
        if money == None:
            money = input("{} , how much pocket money do you
get?".format(self.getName()))
            self.__pocketMoney = money

    def getPocketMoney(self):
        '''get pocket money of student'''
        return self.__pocketMoney
```

Use **help** to see the classes' definitions.

```
>>> help(Person)
```

```
>>> help(Student)
```

What do you see?